FACULDADE IBRATEC

# Programação Orientada a Objetos em Java

Prof. Msc Marcos de Souza

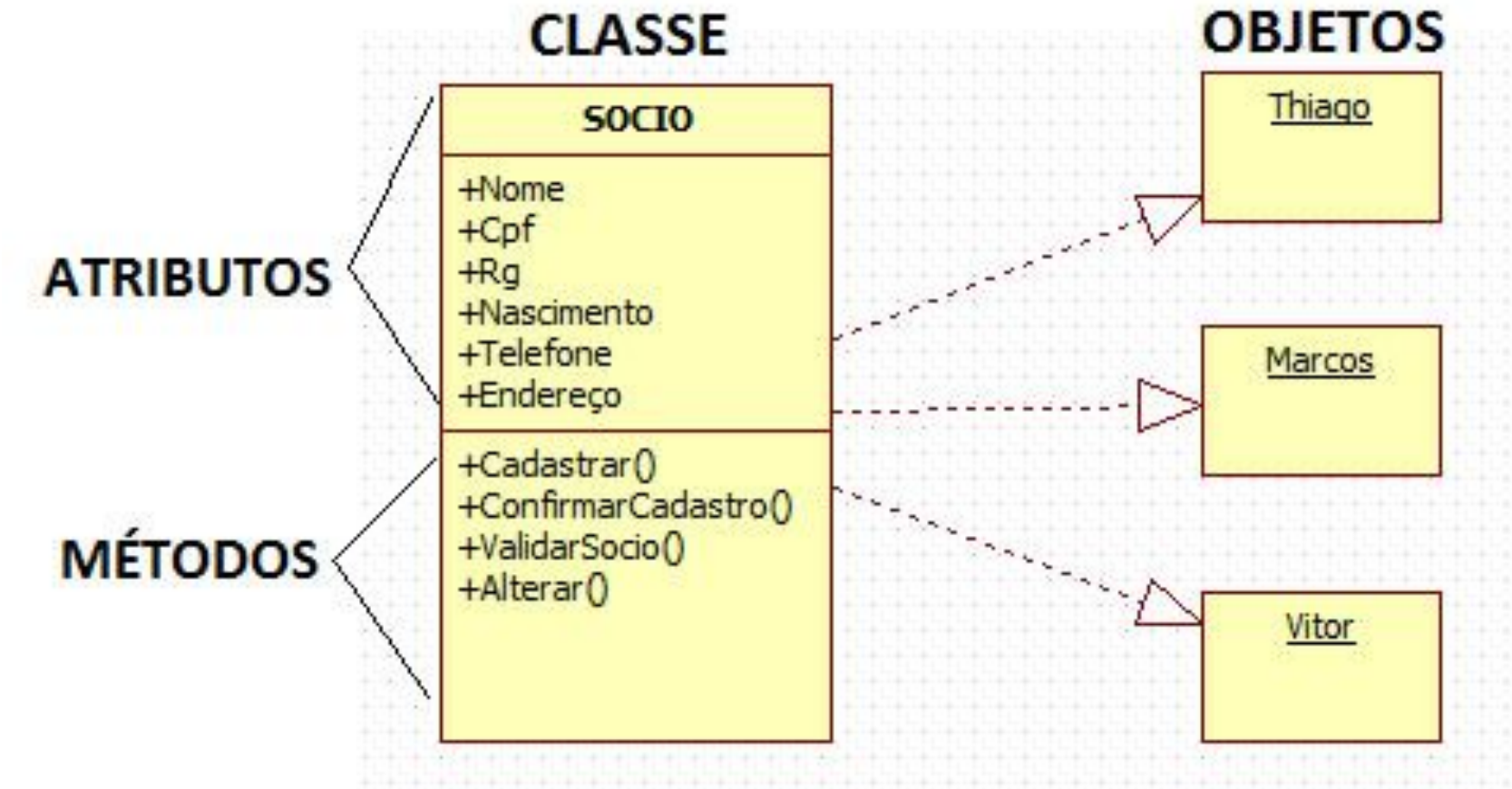
# revisão

FACULDADE IBRATEC

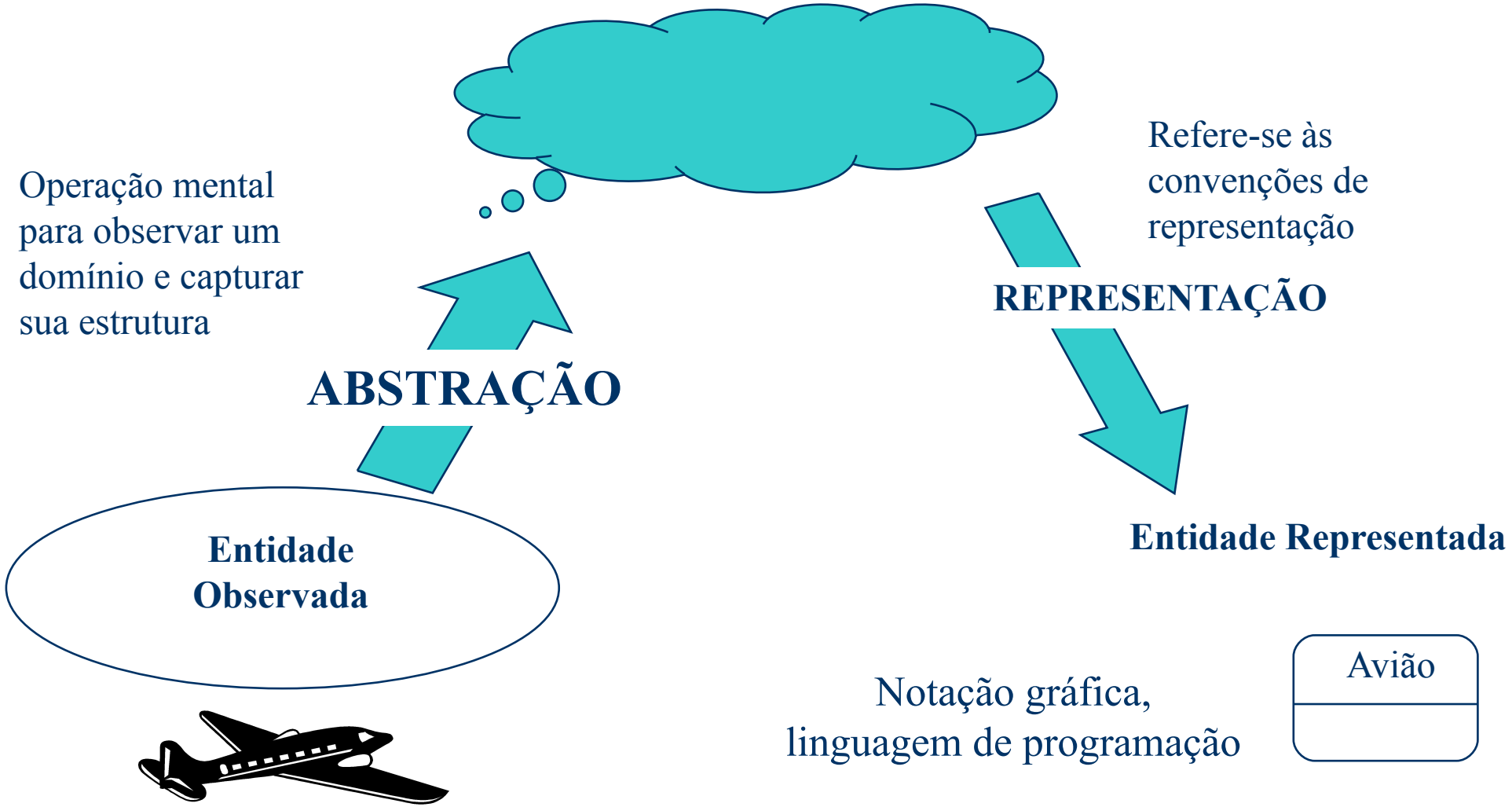
# Assuntos

- Classes e objetos
- Estruturas
- Herança
- Encapsulamento e polimorfismo
- Exceções
- Classes abstratas e Interfaces

# Classes e Objetos



# Modelagem



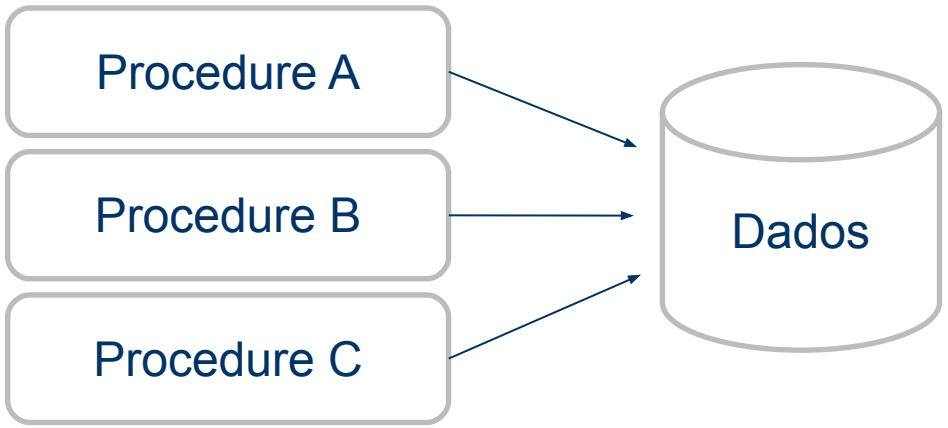

Notação gráfica,
linguagem de programação

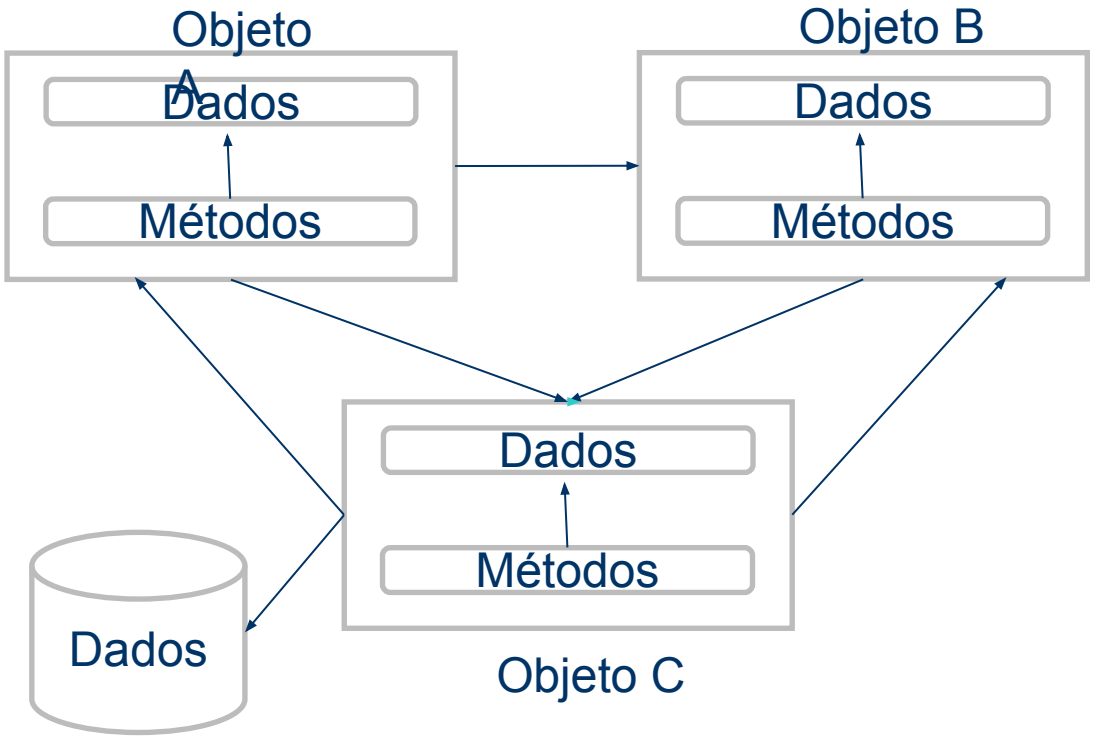



←-- UML

# O Paradigma

- **Paradigma Tradicional X Orientado a Objetos**



# Classes e Objetos

Imagine uma tabela. Onde está a classe? E o objeto?

| Nome | País | Idade | Nota |
|---|---|---|---|
| Renata | BRA | 23 | 3 |
| Marcela | EUA | 25 | 4 |
| Fernanda | ARG | 39 | 2 |
| Caroline | ARG | 28 | 6 |
| Izabela | ALE | 19 | 8 |
| Thais | BRA | 30 | 9 |

# Estruturas

FACULDADE IBRATEC

# if/else

```
if (condition) {
    // code
}
```

```
if (condition) {
// code
} else {
// code
}
```

```
if (condition1) {
// code
} else if(condition2){
// code
} else if (condition3) {
// code
} else {
// code
}
```

Como essa condição é definida ?

# Tabela Verdade

| A | B | NOT A | A AND B | A NAND B | A OR B |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 0 | 1 |

# Operador Ternário

```
boolean cond1 = true, cond2 = true, cond3 = true, cond4 = true;
boolean condition = (!(cond1 || cond2) || !cond3 && cond1) && cond4;
System.out.println(condition ? "imprime 1" : "imprime 2");
```

# Switch/case

```
1  public class Tableless {
2
3      public static void main(String[] args) {
4
5          int valor = 0;
6
7          switch (valor) {
8              case 1:
9                  System.out.println("Valor = 1");
10                 break;
11
12             case 2:
13                 System.out.println("Valor = 2");
14                 break;
15
16             case 3:
17                 System.out.println("Valor = 3");
18                 break;
19
20             default:
21                 System.out.println("Nenhum dos 3 valores");
22         }
23     }
24 }
```

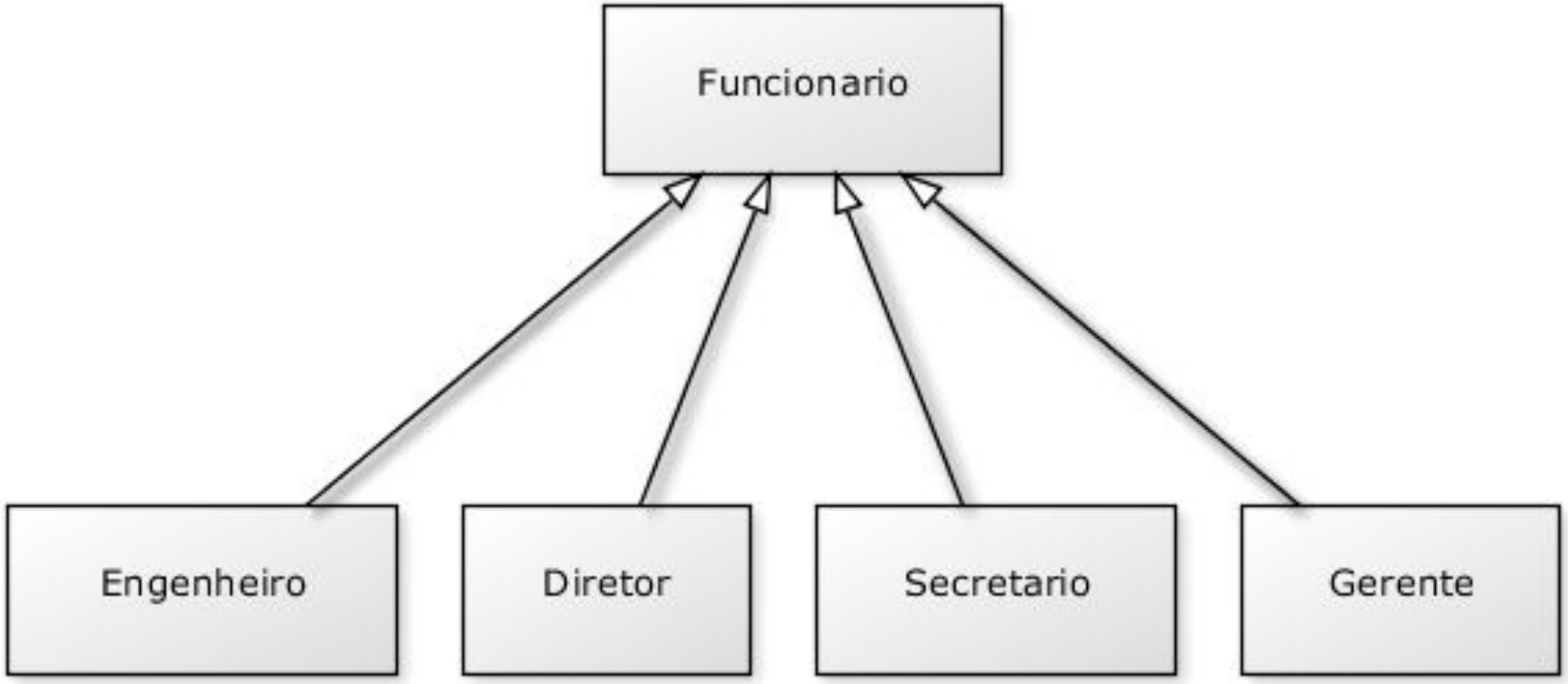
# For

```
String[] strings = {"a", "b", "c", "d", "e", "f"};
String value;

for (int i = 0; i < strings.length; i++) {
        value = strings[i];
        System.out.println("values 1: " + value);
}

for (int i = strings.length-1; i >= 0; i--) {
        value = strings[i];
        System.out.println("values 2: " + value);
}

for (String string : strings) {
        value = string;
        System.out.println("values 3: " + value);
}
```

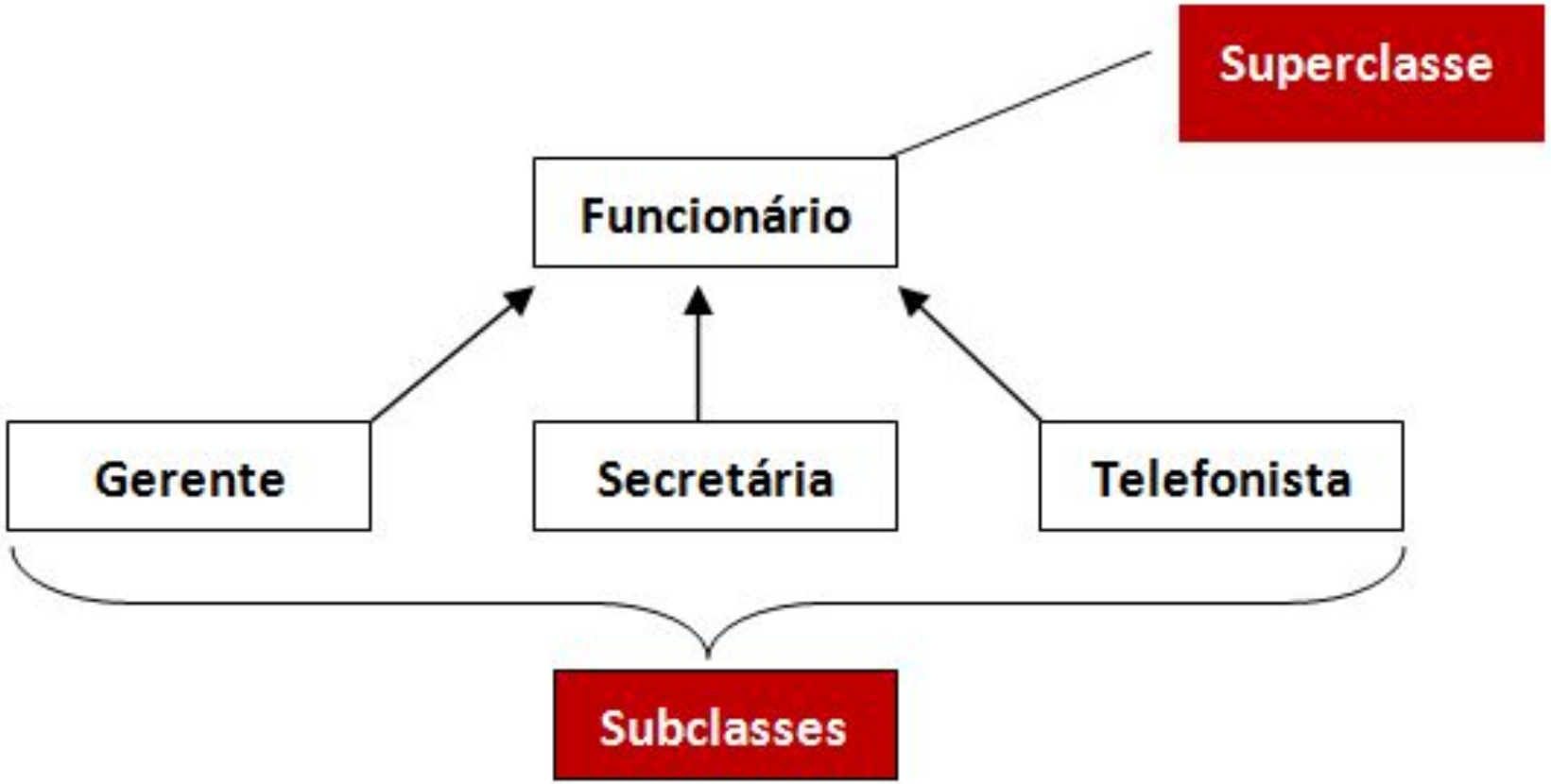
# Herança

FACULDADE IBRATEC

# Herança



# Herança


Superclasse
Funcionário
Gerente
Secretária
Telefonista
Subclasses

# Herança

Item.java

```java
public class Item
{
    private boolean titulo;
    private boolean duracaoTotal;
    private boolean comentario;

    // MÉTODOS OMITIDOS
```

CD.java

```java
public class CD extends Item
{
    private boolean artista;
    private boolean qtdFaixas;

    //MÉTODOS OMITIDOS
```

Para criar uma **subclasse**, basta adicionar a palavra **extends** e o nome da **superclasse**.

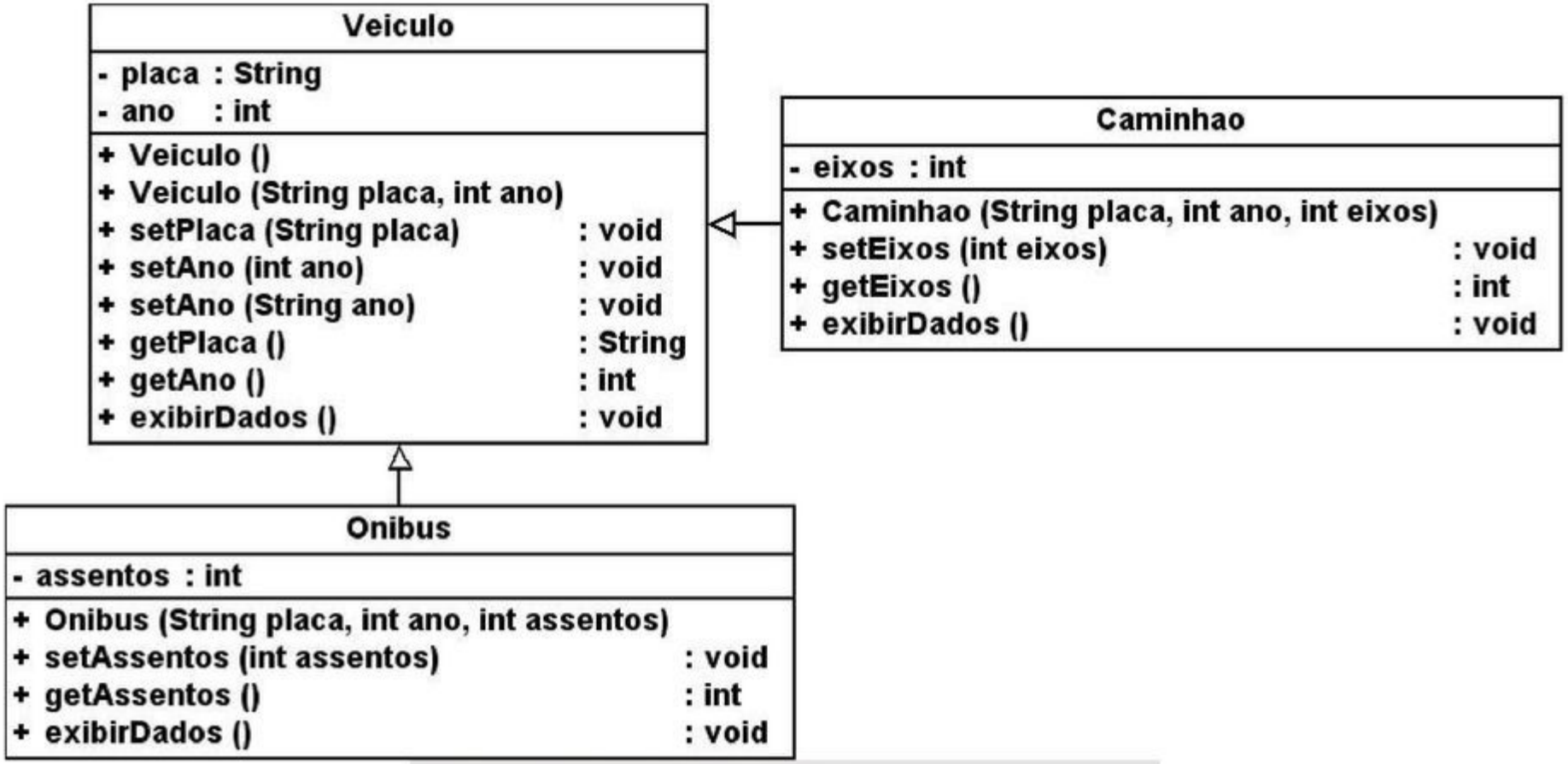
# Polimorfismo

FACULDADE IBRATEC

# Polimorfismo

Conceito utilizado em POO para ilustrar o uso de herança de classes, onde as subclasses sobrescrevem/implementam os métodos da superclasse.

# Polimorfismo



# Encapsulamento

FACULDADE IBRATEC

# Encapsulamento

Conceito que se refere a como a informação é disponibilizada.

- Precisamos tomar cuidado para não disponibilizar códigos para métodos/atributos que não são chamados externamente

# Encapsulamento

```
7   public class Pessoa {
8
9       private String nome;
10      private String cpf;
11
12      public Pessoa(String pName, String pCpf){
13          this.nome = pName;
14          this.cpf = pCpf;
15      }
16
17      public String getNome() {
18          return nome;
19      }
20
21      public void setNome(String nome) {
22          this.nome = nome;
23      }
24
25      public String getCpf() {
26          return cpf;
27      }
28
29      public void setCpf(String cpf) {
30          this.cpf = cpf;
31      }
32
33  }
```

Classe Pessoa

Atributos da classe

Construtor de classe

Método get... recupera atributo

Método set... atribui valor

# Encapsulamento

```
class Cliente {
  private String nome;
  private String endereco;
  private String cpf;
  private int idade;

  public void mudaCPF(String cpf) {
    validaCPF(cpf);
    this.cpf = cpf;
  }

  private void validaCPF(String cpf) {
    // série de regras aqui, falha caso não seja válido
  }
```

Aqui o método validaCPF
não se faz necessidade
de ser acessado externamente

# Exceções

FACULDADE IBRATEC

# Exceções

Exception

Exceções de usuário

IOException

- Erros de I/O
- URL

RuntimeException

- Erro de Cast
- Limites de array
- NullPointer

# Exceções

```
public class SaldoInsuficenteException
    extends Exception {

  private String numero;

  public SaldoInsuficienteException(String numero) {
    super("Saldo insuficiente!");
  }

}
```

# Exceções

- Todo método que pode gerar uma situação de erro pode levantar uma exceção;
- Exceções são ***declaradas*** na assinatura dos métodos usando **throws;**
- Exceções são ***lançadas*** no corpo dos métodos usando o comando **throw:**

```
void metodo(...) throws Exc1, Exc2, ExcN {
  ...
  throw new Exc1(...);
}
```

# Exceções

- Criação explícita de exceções

```
public class Conta{
  ...

 public void debitar(double valor)
    throws SaldoInsuficienteException {

  if (valor <= saldo) {
    saldo = saldo - valor;
  }else {
   throw new
   SaldoInsuficienteException(getNumero());
  }
}
```

# Exceções

- Exceções são tratadas usando blocos: **try-catch [-finally]**

- O **finally** é opcional e sempre é executado, independente se aconteceu exceção ou não.

```
try {
 // chamada aos métodos que podem lançar
 // exceções
} catch (Exc1 ex) {
  // código para tratar um tipo de exceção
} finally {
   //codigo que vai ser sempre executado
 }
```

# Exceções

```
try {
  contas.debitar("123-4",90.00);
  System.out.println("Débito efetuado");
}catch (SaldoInsuficienteException ex) {
 System.out.println(ex.getMessage());
 System.out.println(ex.getNumero());
}catch(ContaInexistenteException ex) {
  System.out.print(ex.getMessage());
  ex.printStackTrace();
}finally{
  System.out.print(“try executado”);
}
```

# Exceções

- Procure tratar diferentes exceções num só bloco

```
try {
 //codigo que levanta Exc1
}catch (Exc1 ex) {...}

try {
 //codigo que levanta Exc2
}catch (Exc2 ex) {...}

try {
 //codigo que levanta Exc3
}catch (Exc3 ex) {...}
```

```
try {
 //codigo que levanta Exc1
 //codigo que levanta Exc2
 //codigo que levanta Exc3
}catch (Exc1 ex) {...}
}catch (Exc2 ex) {...}
}catch (Exc3 ex) {...}
```

# Exceções

- Trate primeiro as exceções específicas

```
try {
 //codigo que levanta SaldoInsuficienteException
 //codigo que levanta ContaInexistenteException
}catch (SaldoInsuficienteException ex) {
 ...
}catch (ContaInexistenteException ex) {
 ...
}
}catch (Exception ex) {
 ...
```

# Classes Abstratas e Interfaces

FACULDADE IBRATEC

# Classes Abstratas

- Não podem ser instanciadas
- São utilizadas para definição de template de classes
- Os métodos abstratos são obrigados a serem implementados

# Classes Abstratas

```
public abstract class Conta {
   protected double saldo;

   public void retira(double valor) {
      this.saldo -= valor;
   }

   public void deposita(double valor) {
      this.saldo += valor;
   }

   public double getSaldo() {
      return this.saldo;
   }

   public abstract void atualiza();
}
```

# Interfaces

- Como criar a classe **CirculoDesenhavel** que é ao mesmo tempo **Circulo** e **Desenhavel**?

```
public class CirculoDesenhavel
      extends Circulo, Desenhavel { ... }
```

- Mas o código acima está ERRADO! Pois Java não suporta herança múltipla.

# Interfaces

- Para resolver esse tipo de problema, Java define o conceito de *interfaces.*

- Uma interface é semelhante a uma classe abstrata, exceto pelo fato de usar a palavra-chave *interface* no lugar de *abstract class* na definição da classe.

# Interfaces

```
public interface Autenticavel {
   public boolean autentica(int senha);
}
```

```
class Gerente extends Funcionario
                        implements Autenticavel {
   private int senha;

   // outros atributos e métodos
   public boolean autentica(int senha) {
      if (this.senha != senha) {
         return false;
      }
      // pode fazer outras possiveis verificacoes,
      // como saber se esse departamento do
      // gerente tem acesso ao Sistema
      return true;
   }
}
```

# Projetos

FACULDADE IBRATEC

# Projetos

- Catálogo de livros digitais e físicos
- Catálogo de pratos de um restaurante
- Estoque de produtos
- Sistema de autenticação (com cadastro de usuários)
- Carro autônomo
- Sistema de entrega de água
- Agenda de compromissos digital
- Chatbot de SAC simples